

Aula 0: Preparação

Faça isso antes do primeiro encontro. Instalar Ruby, Docker e criar contas leva tempo, e é o que mais atrasa capacitação. Se travar em algum passo, manda mensagem no grupo: não espere o dia.

Ao final você deve conseguir rodar os quatro comandos da seção [Checagem final](#).

1. Se você usa Windows: instale o WSL2 primeiro

Ruby e Rails rodam mal no Windows nativo. A forma suportada é o **WSL2** (um Linux de verdade dentro do Windows). Todo o resto deste guia você fará **dentro do WSL2**, não no PowerShell.

No PowerShell **como administrador**:

```
wsl --install -d Ubuntu-24.04
```

Reinicie o computador. Ao abrir o Ubuntu pela primeira vez, ele pede um usuário e uma senha, anote a senha, ela é o seu `sudo`.

A partir daqui, "terminal" significa **o terminal do Ubuntu (WSL2)**.

Se você usa Linux ou macOS, pule esta seção.

2. Git

```
# Ubuntu / WSL2
sudo apt update && sudo apt install -y git curl build-essential

# macOS (instala as ferramentas de linha de comando da Apple, que incluem o git)
xcode-select --install
```

Configure seu nome e e-mail. Eles vão em todo commit que você fizer:

```
git config --global user.name "Seu Nome"
git config --global user.email "seu@email.com"
```

3. Ruby 3.4.9 via mise

O `mise` é um gerenciador de versões: ele instala a versão exata de Ruby que o projeto pede, sem mexer no Ruby do sistema. É o equivalente do `pyenv` do mundo Python.

Por que não `apt install ruby`? Porque a versão do sistema é antiga e compartilhada. Se dois projetos pedirem versões diferentes, você fica sem saída. `mise` resolve isso por projeto.

Instalar o mise

```
curl https://mise.run | sh
echo 'eval "$(${~/.local/bin/mise activate bash})"' >> ~/.bashrc
exec bash
```

Usa `zsh` (padrão do macOS)? Troque as duas ocorrências de `bash` por `zsh` e o `~/.bashrc` por `~/.zshrc`.

Instalar as dependências de compilação do Ruby

```
# Ubuntu / WSL2
sudo apt install -y autoconf patch rustc libssl-dev libyaml-dev libreadline-dev \
  zlib1g-dev libgmp-dev libncurses-dev libffi-dev libgdbm-dev libdb-dev uuid-dev \
  libpq-dev libvips

# macOS (precisa do Homebrew: https://brew.sh)
brew install openssl@3 readline libyaml gmp libpq vips
```

Instalar o Ruby

```
mise use --global ruby@3.4.9
```

Isso compila o Ruby do zero: leva de **5 a 15 minutos**. É normal.

```
ruby -v      # ruby 3.4.9 ...
gem -v
```

Instalar o Rails

```
gem install rails -v 8.1.3
rails -v      # Rails 8.1.3
```

4. Docker

O banco de dados (PostgreSQL) vai rodar dentro de um container Docker, em vez de instalado na sua máquina. Assim todo mundo tem exatamente a mesma versão, e desinstalar é apagar um container.

- **Windows/WSL2 e macOS:** instale o [Docker Desktop](#). No Windows, nas configurações, habilite a integração com a distro `Ubuntu-24.04`.
- **Linux:** siga o [guia oficial](#) e depois rode `sudo usermod -aG docker $USER` e **saia e entre de novo na sessão**.

Testando:

```
docker run --rm hello-world
```

Se aparecer "Hello from Docker!", está pronto.

5. VS Code e extensões

Baixe em code.visualstudio.com. No Windows, instale no **Windows** (não dentro do WSL): ele se conecta ao WSL sozinho.

Extensões (Ctrl+Shift+X e busque pelo nome):

Extensão	Para quê
Ruby LSP (Shopify)	Autocomplete, ir para definição, erros em tempo real
Ruby Rails (Aki)	Navegação entre model/controller/view
Docker (Microsoft)	Ver e gerenciar containers pela barra lateral
WSL (Microsoft)	<i>Só Windows:</i> abrir projetos do Ubuntu no VS Code
GitLens	Ver quem escreveu cada linha e quando
vscode-icons	Ícones por tipo de arquivo (a mesma da capacitação de front)

6. Insomnia (ou Postman)

Nossa API não tem tela: ela responde JSON. Para chamar as rotas, usaremos um cliente HTTP. Baixe o [Insomnia](#) (mais simples) ou o [Postman](#), o que preferir.

7. Contas

GitHub

Crie uma conta em github.com se ainda não tem. É só isso: a capacitação não usa nada pago do GitHub.

Instale o **GitHub CLI**, que na Aula 1 publica o seu projeto num comando:

```
# Ubuntu / WSL2
sudo apt install -y gh
# macOS
brew install gh

gh auth login      # escolha GitHub.com → HTTPS → login pelo navegador
gh auth status     # tem que dizer "Logged in to github.com"
```

Configure também o acesso por SSH ao GitHub, se ainda não tem:

```
ssh-keygen -t ed25519 -C "seu@email.com"    # Enter em tudo
gh ssh-key add ~/.ssh/id_ed25519.pub --title "meu-notebook"
ssh -T git@github.com                       # tem que cumprimentar você pelo nome
```

O servidor de SSH: teste isso com antecedência

Na Aula 4 a **sua própria máquina** vira o servidor onde a API vai rodar. Para isso ela precisa aceitar conexão SSH, e o programa que faz isso é o `openssh-server`. Você já usa o *cliente* de SSH (é o comando `ssh`); o que falta é o *servidor*.

Se você usa Windows, tudo abaixo é **dentro do WSL2**, que é o seu Linux.

```
# Ubuntu, Debian e WSL2
sudo apt update && sudo apt install -y openssh-server
sudo service ssh start

# Fedora
sudo dnf install -y openssh-server && sudo systemctl enable --now sshd
```

No **macOS** não se instala nada: ligue em **Ajustes do Sistema** → **Geral** → **Compartilhamento** → **Sessão remota**.

Confira, conectando na sua própria máquina:

```
ssh "$USER"@127.0.0.1 'echo FUNCIONOU'
```

Ele vai pedir a sua senha (na Aula 4 isso vira uma chave) e tem que responder `FUNCIONOU`. **Mande o resultado no grupo**. Dez minutos hoje valem a aula inteira.

WSL2: o `sshd` não sobe sozinho a cada boot do Windows, a não ser que você habilite o systemd. Se um dia o `ssh` parar de conectar do nada, é isso, e a saída é `sudo service ssh start`.

Clone o repositório da capacitação

Você vai usá-lo o curso inteiro, para consultar o código pronto:

```
git clone <url-do-repositório> ~/capitacao-gabarito
```

Checagem final

Cole os comandos no terminal. Se todos responderem, você está pronto:

```
ruby -v           # ruby 3.4.9
rails -v          # Rails 8.1.3
docker -v         # Docker version 2x.x.x
git --version
gh auth status    # Logged in to github.com
ssh "$USER"@127.0.0.1 'echo ok' # ok (é o servidor da Aula 4)
```

Como a capacitação funciona

Você vai trabalhar em **dois diretórios**:

```
~/capitacao-gabarito/  ← o repositório da capacitação. Só leitura.
~/automic_auth_api/    ← o SEU projeto. Criado na Aula 1, é onde você escreve.
```

O gabarito você já clonou, alguns passos acima:

```
ls ~/capitacao-gabarito # se não existir:
# git clone <url-do-repositório> ~/capitacao-gabarito
```

O seu projeto você cria no primeiro encontro, com o passo a passo de `01-fundamentos.md`. Ele precisa ficar no **seu** GitHub: na Aula 4 é de lá que o CI roda e é para a sua conta que a imagem Docker vai.

Se algo deu errado

Sintoma	Provável causa e saída
<code>mise: command not found</code>	A linha do <code>activate</code> não foi para o arquivo certo. Confira <code>~/.bashrc</code> (ou <code>~/.zshrc</code>) e rode <code>exec bash</code> .
A instalação do Ruby falha com erro de compilação	Faltou alguma dependência do passo 3. Rode o <code>apt install / brew install</code> de novo e tente <code>mise install ruby@3.4.9</code> outra vez.
<code>permission denied</code> no <code>docker</code>	Você não está no grupo <code>docker</code> . Rode <code>sudo usermod -aG docker \$USER</code> e saia e entre de novo (só reabrir o terminal não basta).
No Windows, o <code>docker</code> não aparece dentro do Ubuntu	Docker Desktop → Settings → Resources → WSL Integration → habilite a distro <code>Ubuntu-24.04</code> .
<code>ssh \$USER@127.0.0.1</code> dá <code>Connection refused</code>	O <code>sshd</code> não está rodando: <code>sudo service ssh start</code> . No macOS, ligue a Sessão remota.
<code>gem install rails</code> reclama de permissão	Você está usando o Ruby do sistema, não o do <code>mise</code> . Confira com <code>which ruby</code> : deve apontar para dentro de <code>~/.local/share/mise</code> .

Mais casos em [troubleshooting.md](#).